

Translated

From: e-maxx.ru

Linear Sieve

Given a number n , find all prime numbers in a segment $[2;n]$.

The standard way of solving a task is to use [the sieve of Eratosthenes](#). This algorithm is very simple, but it has runtime $O(n \log \log n)$.

Although there are a lot of known algorithms with sublinear runtime (i.e. $O(n)$), the algorithm described below is interesting by its simplicity: it isn't any more complex than the classic sieve of Eratosthenes.

Besides, the algorithm given here calculates **factorizations of all numbers** in the segment $[2;n]$ as a side effect, and that can be helpful in many practical applications.

The weakness of the given algorithm is in using more memory than the classic sieve of Eratosthenes: it requires an array of n numbers, while for the classic sieve of Eratosthenes it is enough to have n bits of memory (which is 32 times less).

Thus, it makes sense to use the described algorithm only until for numbers of order 10^7 and not greater.

The algorithm is due to Paul Pritchard. It is a variant of Algorithm 3.3 in (Pritchard, 1987: see references in the end of the article).

Algorithm

Our goal is to calculate **minimum prime factor** $lp[i]$ for every number i in the segment $[2;n]$.

Besides, we need to store the list of all the found prime numbers - let's call it pr .

We'll initialize the values $lp[i]$ with zeros, which means that we assume all numbers are prime. During the algorithm execution this array will be filled gradually.